

Concurrent Socket Server

Zach Gray, Genesis Dingle, and Fazal rahman Safa

CNT4504 – Computer Networks & Distributed Processing

Professor Scott Kelly

Introduction

The main purpose of this project is to create and demonstrate the performance of a concurrent multi-threaded server that can handle multiple client connections at the same time in a client-server network model. In this project, we analyze how the server affects turnaround time as it processes client requests. The goal is to design the client and server so they can support several basic operations, such as retrieving the current date and time, uptime, memory usage, network status, current users, and running processes. By running these operations under different levels of load, we can see how well the concurrent server handles an increasing number of requests. In the later sections of this report, we describe how the client and server were set up and configured, explain how we collected our data, and analyze how increasing the number of clients influenced the turnaround time for each operation. Lastly, we will cover our key findings and discuss the lessons we learned while building and testing the concurrent server.

Client server setup and configuration

The client and the server were implemented through Java using the Java libraries. The project has two main programs `ConcurrentServer.java` which implements the multi-threaded server using a `ServerSocket` and a fixed thread pool, and `Main.java`, which implements the client that is capable of spawning many concurrent requests using an `ExecutorService` and `ClientWorker` threads. The client and server together create a working system to handle all of the function calls we must request. Starting off with the server, we run it by calling `java ConcurrentServer <port>` this creates our `ServerSocket` and allows the program to listen for

incoming connections. We then move to the start method, where the server waits for a connection, accepts it, and immediately submits a request to the thread pool so that a worker can handle the request while the main thread goes back to accepting more clients. Once we have received a connection, the worker thread calls the `handleClient()` method and uses a `BufferedReader` to read the request that was sent from the client. We then interpret the request using a `PrintWriter` to send the response back. Our server then moves to the `processCommand()` method, where it matches what was requested from the client and then executes the request through our appropriate helper method using `ProcessBuilder`, collects the command output into a response string, and sends the information it gathers back to the client before closing the connection.

Now moving over to the Client code called `Main.java`, we start by requesting the host address, port number, command you want to run, and the number of times it wants to execute the command. We move to `ExecutorService` executor which creates a number of threads for each of our `List<ClientWorker>` clients: each of the workers creates a connection to the server, sends out one of the commands, and then we read the reply the server gave us using a `BufferedReader`. Each time this process is completed, the Client records its turnaround time using `TimeMillis`. Once all commands have been run, the client takes the results and gets the total average turnaround time.

Testing and data collection

The Concurrent server was tested through running the client code and server code

through Bitvise through hooking up to the UNF VPN and connecting to the local network to be able to find the servers access point. Once connected through Bitvise the commands were run through the console as we were prompted with the server address and port number and then we were hooked in so we would be able to run the data, uptime, memory, netstat, users, and processes commands. We then ran each command 1, 5, 10, 15, 20, 25, and 100 times and then recorded our data we collected being the total turnaround time and average turnaround time of each. After each time we collected data, we recorded the data into a chart which can be found at the end of the report for each of the 6 commands total turnaround time and average turnaround time.

Data Analysis

When analyzing the data that was collected, we can see many things like how in the collection of data for date, uptime, and memory commands they performed overall faster than the other three commands using very little resources. In the concurrent server results, these commands stayed relatively fast for smaller numbers of clients, but their average turnaround time started to increase as more clients were added, especially at higher client counts. For netstat, users, and processes these commands took a heavier toll on the turnaround time as we increased the number of client requests we made. This increase in time mainly occurred because these commands required more resources from the system and required additional processing, which caused delays as the server started handling more requests. When we compare both servers to each other, we can see that the Iterative server handled the lighter commands faster

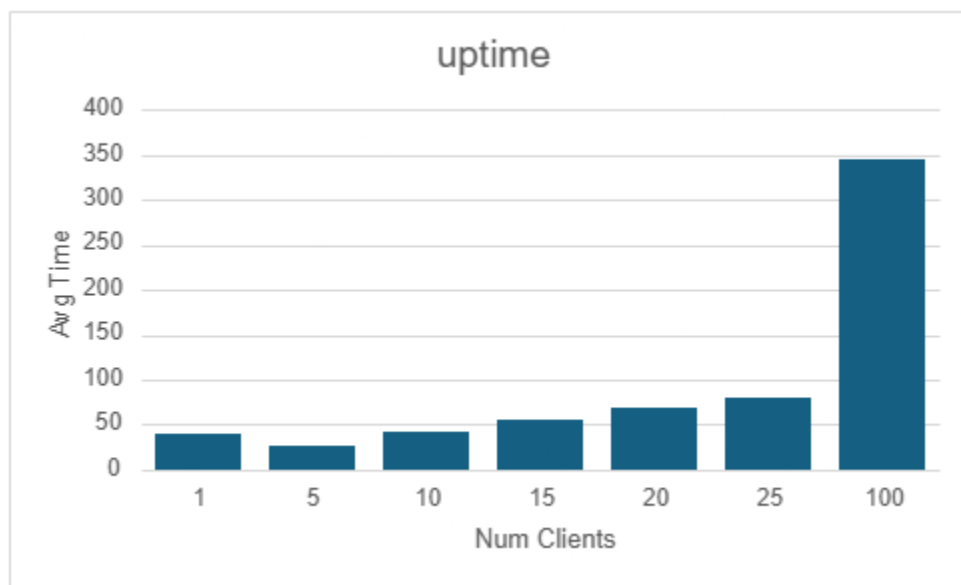
overtime as requests increased. On the concurrent server side, the average overall for all the commands was just higher than the iterative socket server. This mainly shows that concurrent servers can handle many clients at the same time, but it costs a higher average turnaround time. The Iterative Server is better suited for situations where there are only a few clients, and the functions it will perform are lighter. For a concurrent server, it would be best suited when there are many clients who may connect at the same time or maybe when we use heavier functions that need to be handled.

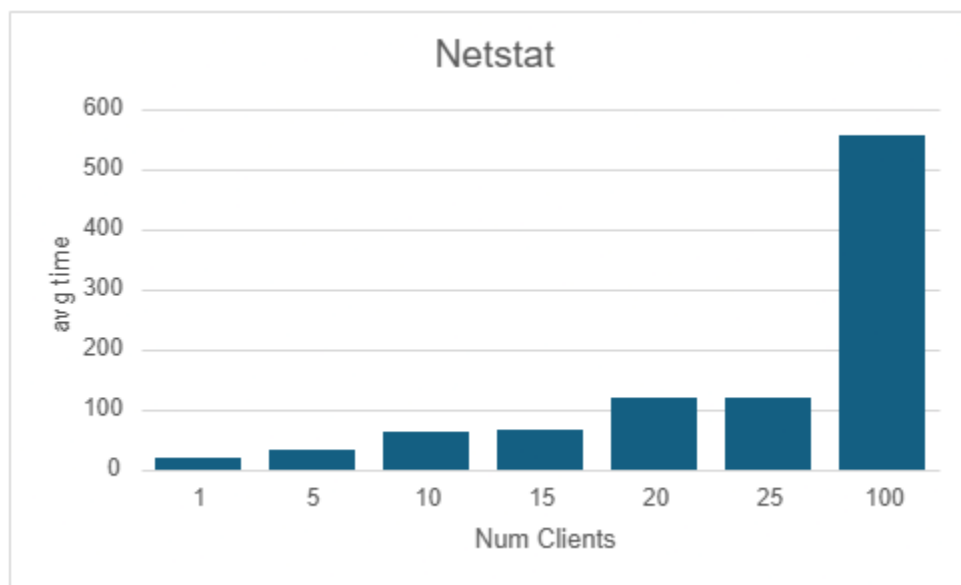
Conclusion

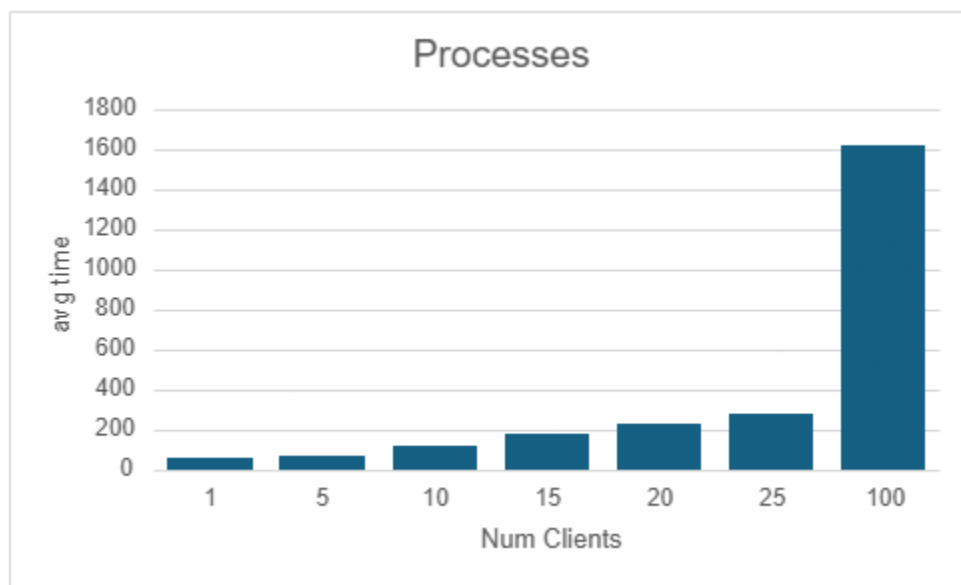
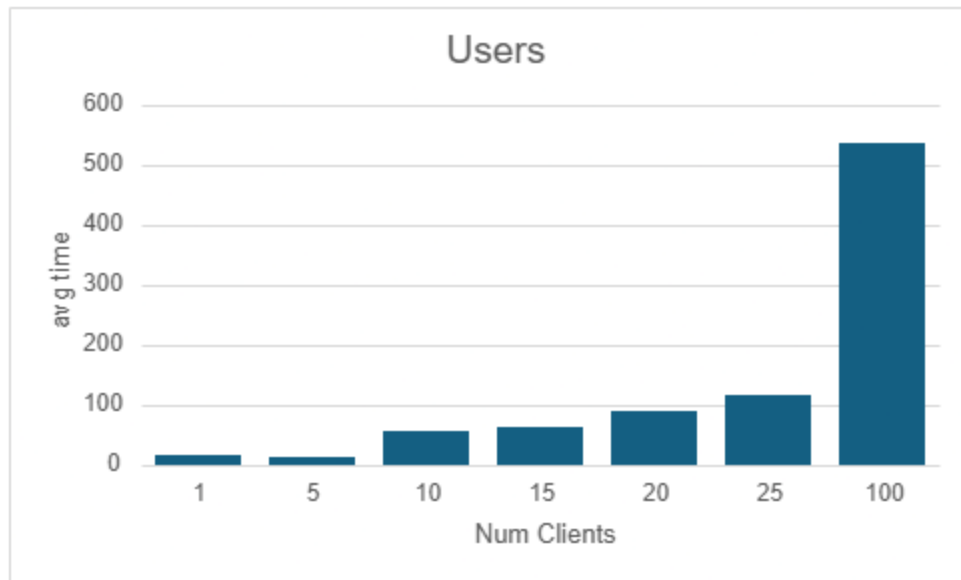
From the results that we found in the project, we can see that the server worked on handling the tasks that were assigned as we handled multiple client connections and processed all of the six commands that were required and collected data on them. From analyzing the data, we can see that lightweight commands like date, uptime, and memory started with very low turnaround times for small numbers of clients and stayed relatively low compared to the heavier commands, even though their average time did increase as more clients were added. This happened mostly because these commands did not require as many resources as the other three commands do, which allowed it to keep a fast turnaround time. For netstat, users, and processes, we found that these commands are heavier than the other three as they require more internal resources which made them have a higher turnaround time even with small increases to client requests.

Lesson learned

Lastly, some things I learned through the project were how to run and design a multi-threaded server. I also learned or really got more familiar with working with the clients that helped run the code. Also, by looking at both the Iterative and concurrent socket code and output you can see many similarities and differences between them. On the Iterative socket server side when comparing results to the concurrent socket server, the lighter commands average turnaround time was faster over time and for the heavier commands the average increased over time. In the concurrent socket server, we saw that for all commands average time was higher everywhere as we increased requests.







Iterative Socket Server

